Security Twin per Spotlighting: Simulazione Monte Carlo e Intrusion Graph

Francesco Polperio, 635406

29 dicembre 2025

Sommario

In questo elaborato descrivo un *Security Twin* per stimare il rischio di compromissione in scenari di *indirect prompt injection* su task di *summarization*, facendo riferimento al framework Spotlighting. Il codice implementa: (i) un benchmark statico di *Attack Success Rate* (ASR) a singolo tentativo; (ii) un modello dinamico di sessione con fino a K tentativi; (iii) metriche di rischio cumulativo e passi medi al compromesso; (iv) un *intrusion graph* che visualizza la progressione degli stati nel caso *Delimiting*.

1 Obiettivo e contesto

L'obiettivo è confrontare la probabilità di compromissione di un LLM sotto diverse difese Spotlighting. Nel mio caso considero:

- **None**: nessuna difesa.
- **InstructionsOnly**: solo istruzioni difensive nel prompt.
- **Delimiting**: separazione rigida tra documento e istruzioni (con un modello toy di *leak*).
- **Datamarking**: confronto riportato nei plot (ASR statico e rischio di sessione).
- **Encoding**: assunta qui come perfetta nel toy model (probabilità zero).

Il codice prende in input valori di ASR (misurati o tratti da letteratura) e costruisce un gemello digitale probabilistico per analizzare il rischio *di sessione* (multi-turn).

2 Configurazione e parametri

2.1 Parametri globali

- `SEED`: seme per la riproducibilità.
- `NUM_SIMULATIONS`: numero di episodi Monte Carlo.
- `MAX_ATTEMPTS` (K): massimo numero di tentativi (turni) in una sessione.

2.2 Modello toy del leak per Delimiting

Nel caso **Delimiting** modello un possibile *leak* (l'attaccante apprende/aggira il delimitatore) con un processo geometrico con probabilità per-step:

$$P_{\text{leak}} = 1 - e^{-\lambda}$$

dove `LEARN_LAMBDA` rappresenta la velocità media di apprendimento nel tempo. Dopo il leak, nel worst-case considero la difesa aggirata e uso il comportamento *baseline*.

3 Input ASR (statico)

Il dizionario `MODELS_ASR` contiene per ciascun modello e difesa una stima della probabilità di successo dell'attacco in un singolo tentativo (ASR):

$$\mathrm{ASR}_{\mathrm{static}}(\mathrm{model}, \mathrm{defense}) \in [0, 1].$$

Questi valori sono usati come *parametri* del twin (non sono ricalcolati automaticamente dal codice).

3.1 Stima manuale degli ASR per Qwen e Mistral (Ollama)

I valori di ASR per **Qwen 2.5** e **Mistral** sono stati stimati manualmente tramite esecuzioni locali con **Ollama**. Per ciascuna difesa (`None`, `InstructionsOnly`, `Delimiting`, `Datamarking`, `Encoding`) ho effettuato **20 tentativi**, mantenendo **gli stessi prompt di attacco** e lo stesso task (stesso documento), cambiando solo la componente di difesa nel prompt (delimitatori, istruzioni, encoding, ecc.) e ripetendo l'attacco in modo coerente tra le condizioni.
L'ASR empirico è:

$$\widehat{\mathrm{ASR}} = \frac{\#\mathrm{successi}}{20}.$$

Questi valori servono poi come input del Security Twin per passare dal caso statico (1 tentativo) al rischio dinamico di sessione (fino a K tentativi).

4 Security Twin dinamico

4.1 Definizione del rischio di sessione

Nel caso dinamico, ogni episodio simula una sessione con fino a K tentativi. La sessione è compromessa se *almeno un tentativo* ha successo:

$$P(\text{compromesso in sessione}) = 1 - \prod_{t=1}^{K} (1 - p_t)\,,$$

dove p_t può cambiare nel tempo in base allo stato (ad esempio, dopo un leak).

4.2 Stato del twin

Per Delimiting lo stato concettuale è:

$$s_t = (t, \mathrm{leaked}).$$

Nel codice lo rappresento in forma estesa come tupla `(t, delim_leaked, token_guessed)` per compatibilità generale con più difese.

4.3 Logica per Delimiting: leak e collasso a baseline

- Finché non avviene il leak, la probabilità per tentativo è p_{def}.
- Dopo il leak, nel worst-case uso p_{base} (difesa aggirata).

Nota sulla coerenza (anti-paradosso). Per alcuni modelli una difesa può risultare empiricamente peggiore del baseline. Nel codice è presente una regola che evita il paradosso in cui l'evento di "apprendimento" riporterebbe a un baseline più sicuro.

4.4 Logica per Datamarking: Token Guessing

Mentre per il Delimiting il rischio principale è l'apprendimento del delimitatore statico (leak nel tempo), per il **Datamarking** assumiamo l'implementazione raccomandata di *dynamic marking*. In questo scenario, il token di marcatura non è fisso ma viene rigenerato a ogni invocazione (o sessione) scegliendo casualmente un k-gramma da un alfabeto di dimensione N.

La variabile di stato `token_guessed` presente nella tupla modella la probabilità che l'attaccante riesca a indovinare il token corrente per "falsificare" la continuità del segnale nel prompt. La probabilità di successo dell'attacco (guessing) in un singolo tentativo è approssimata da:

$$P_{\text{guess}} = \frac{1}{N^k}$$

Se l'evento di guessing si verifica (catturato dalla variabile `token_guessed`), la difesa viene considerata compromessa per quel tentativo.

4.5 Output del twin

La funzione `run_dynamic_twin` restituisce:

- `dyn_rate`: frazione di episodi compromessi (rischio di sessione).
- `steps_to_compromise`: lista dei passi t in cui avviene il primo successo.
- `hacked_by_step[t]`: conteggio compromessi avvenuti esattamente al passo t.
- `traces`: transizioni stato-azione-stato, usate per il grafo (solo per Delimiting).

5 Metriche derivate e grafici

5.1 Passi medi al compromesso

Se esistono compromissioni osservate:

$$\overline{T} = \frac{1}{m} \sum_{i=1}^{m} T_i,$$

dove T_i è il passo del compromesso nel i-esimo episodio compromesso.

5.2 Rischio cumulativo per step

Il rischio cumulativo fino allo step t è:

$$\widehat{P}_t = \frac{\sum_{j=1}^{t} \text{hacked_by_step}[j]}{N}.$$

Il codice produce un grafico con il confronto *Static vs Dynamic* per ciascuna difesa, e una curva di evoluzione del rischio cumulativo nel multi-turno.

6 Intrusion Graph per Delimiting

6.1 Motivazione

L'intrusion graph rende esplicita la dinamica degli stati nella sessione. I nodi sono stati e gli archi sono transizioni osservate nelle tracce.

6.2 Layout deterministico (ordinato)

Per evitare grafi "a spaghetti", il layout è una griglia:

- asse x: step temporale t;
- asse y: classe dello stato (CLEAN/LEAK);
- nodo COMPROMISED posizionato a destra.

Le etichette vengono mostrate solo per gli archi più frequenti (e sempre per quelli verso COMPROMISED), così il grafo resta esplicativo senza diventare illeggibile.

Nota. Nel report includo l'intrusion graph solo per `Delimiting`, perché è la difesa in cui la variabile di stato `delim_leaked` introduce una dinamica chiara e interpretabile. Le altre difese restano comunque confrontate nei plot (ASR statico e rischio di sessione).

7 Riproducibilità e output

Il codice salva:

- `security_twin_results_<model>.png`: grafici principali.
- `intrusion_graph_delimiting_<model>.png`: grafo per Delimiting.

8 Discussione dei risultati

Effetto multi-turno. I grafici mostrano che un ASR moderato a singolo tentativo (static, 1 try) può tradursi in un rischio di sessione (dynamic, ≤ 20 try) molto elevato. Assumendo tentativi indipendenti con probabilità costante p, il rischio teorico di compromissione entro K tentativi è:

$$P(\text{compromesso entro } K) = 1 - (1-p)^K.$$

Ad esempio, con $p = 0.15$ e $K = 20$ si ottiene $1 - (0.85)^{20} \approx 96\%$, in linea con l'andamento osservato per Mistral in modalità `None`.

GPT-3.5. Le configurazioni `None` e `InstructionsOnly` hanno un ASR statico elevato ($\approx 60\%$) e convergono rapidamente a rischio di sessione vicino al 100% in pochi tentativi. La difesa `Delimiting` riduce l'ASR statico (ritardo dell'attacco), ma nel modello dinamico il rischio converge comunque verso il 100%: l'effetto principale è un *delay* del compromesso, non la sua eliminazione, coerentemente con la possibilità di leak modellata dal twin. `Encoding` rimane a rischio nullo nel toy model. `Datamarking` (nei plot) mostra che anche un ASR statico basso può accumularsi in sessione e diventare non trascurabile.

Mistral. In `None` un ASR statico $\approx 15\%$ diventa un rischio di sessione $\approx 96\%$ a 20 tentativi. Nel caso `Delimiting`, nonostante ASR statico vicino a zero, il rischio cresce nel tempo perché il twin include un evento di leak (`delim_leaked`) che può riportare la probabilità di successo al baseline (worst-case). Le difese con ASR misurato a zero vanno lette considerando che la stima manuale è su $n = 20$ tentativi.

Qwen 2.5. Si osserva un rischio di sessione molto elevato in quasi tutte le difese entro 20 tentativi. In particolare, `Delimiting` risulta empiricamente peggiore del baseline (ASR statico maggiore), suggerendo un possibile effetto di attenzione/recency indotto dalla struttura del prompt in quel modello e nel set di test. Anche `Datamarking` nei plot mostra che, a fronte di un ASR statico moderato, il rischio cumulativo cresce sensibilmente nel multi-turno.

9 Limitazioni

- Il twin è un modello probabilistico: non sostituisce benchmark reali.
- Il leak di Delimiting è toy (geometrico) e serve a rappresentare in modo interpretabile il peggioramento nel tempo.
- Encoding è trattato come perfetto (ASR=0) per semplicità e confronto.
- Le stime manuali su Ollama sono basate su $n = 20$ tentativi per difesa: il campione è piccolo e può avere varianza non trascurabile.

Nota sull'Indipendenza dei Tentativi. Il presente modello Security Twin assume che, al netto degli eventi di leak (apprendimento strutturale), i singoli tentativi di iniezione siano prove di Bernoulli indipendenti. In uno scenario reale, l'attaccante possiede una "memoria": un fallimento potrebbe fornire informazioni parziali che aumentano la probabilità di successo al tentativo $t + 1$ (apprendimento in-context), oppure diminuirla se il sistema target attiva filtri difensivi adattivi. L'attuale implementazione modella l'apprendimento solo tramite la variabile di stato macroscopica `delim_leaked`.